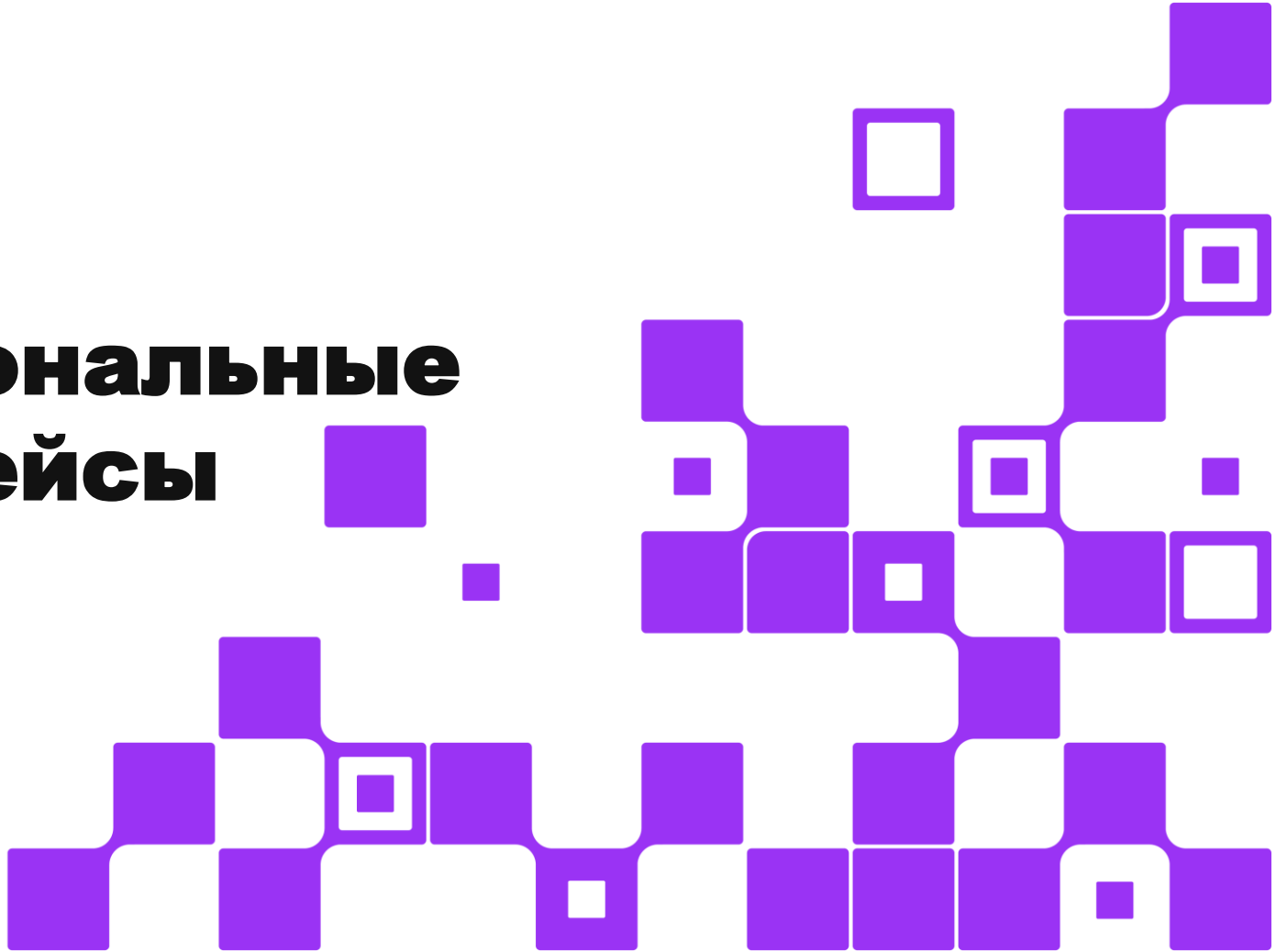
IThub

Функциональные интерфейсы



IThub

Функциональный интерфейс - интерфейс, содержащий ровно один абстрактный метод.

- может быть реализован с помощью лямбда-выражения
- может использоваться со ссылками на методы и конструкторы
- может содержать default- и static-методы
- часто применяется для передачи поведения в методы

Аннотация **@FunctionalInterface** необязательна, но помогает компилятору проверить корректность интерфейса.

IThub

Функциональный интерфейс - интерфейс, содержащий ровно один абстрактный метод.

- может быть реализован с помощью лямбда-выражения
- может использоваться со ссылками на методы и конструкторы
- может содержать default- и static-методы
- часто применяется для передачи поведения в методы

Аннотация **@FunctionalInterface** необязательна, но помогает компилятору проверить корректность интерфейса.

Пакет java.util.function

IThub

Java предоставляет стандартные функциональные интерфейсы в пакете java.util.function.

Они позволяют не создавать собственный интерфейс для каждой типовой операции.

- Predicate<T>
- Function<T, R>
- Consumer<T>
- Supplier<T>
- UnaryOperator<T>
- BinaryOperator<T>

Predicate<T> проверка условия

IThub

- назначение: представляет собой логическую функцию аргумента
- в основном используется для фильтрации данных из потока (stream)
- основной метод: **boolean test(T value)**

```
Predicate<Integer> isEven = n -> n % 2 == 0;
System.out.println(isEven.test(4)); // true
System.out.println(isEven.test(7)); // false
```

```
numbers.stream()
        .filter(n -> n > 0)
        .toList();
```

Function<T, R> преобразование значения

IThub

- назначение: принимает один аргумент и выдает результат
- используется для преобразования типов, обработки объектов, извлечения нужных полей
- основной метод: R apply(T value)

```
Function<String, Integer> length = String::length;
System.out.println(length.apply("Java")); // 4
```

```
names.stream()
    .map(String::toUpperCase)
    .toList();
```

Consumer<T> выполнение действия

IThub

- назначение: принимает значение типа T, выполняет действие и ничего не возвращает
- используется для вывода информации, логирования, отправки уведомлений, обработки каждого элемента коллекции
- основной метод: **void accept(T value)**

```
Consumer<String> printer = System.out::println;
printer.accept("Hello, Java!");
```

```
names.forEach(System.out::println);
```

Supplier<T>
получение значения

IThub

- **назначение: не принимает аргументов и возвращает значение типа T**
- **используется для создания объектов, генерации значений, отложенных вычислений, передачи способа получения данных.**
- **основной метод: T get()**

```
Supplier<Double> randomValue = Math::random;
System.out.println(randomValue.get());
```

```
Supplier<List<String>> listSupplier = ArrayList::new;
List<String> names = listSupplier.get();
```

UnaryOperator<T> и BinaryOperator<T>

Оба интерфейса являются специализированными вариантами Function.

UnaryOperator<T> принимает одно значение типа T и возвращает значение того же типа T

```
UnaryOperator<Integer> square = n -> n * n;
System.out.println(square.apply(5)); // 25
```

BinaryOperator<T> принимает два значения типа T и возвращает одно значение типа T

```
BinaryOperator<Integer> max = Math::max;
System.out.println(max.apply(10, 20)); // 20
```

IThub

Практическое применение

IThub

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6);
List<Integer> result = numbers.stream()
        .filter(n -> n % 2 == 0)
        .map(n -> n * n)
        .toList();

System.out.println(result); // [4, 16, 36]
```

filter() использует Predicate<Integer>
map() использует Function<Integer, Integer>

Задание (КТ №6)

IThub

```
public class Main {
  public static void main(String[] args) {
     Scanner scanner = new Scanner(System.in);
     Main main = new Main();
     Integer n = scanner.nextInt();

     Integer dn = new StateMachine<>(n)
          .get();
     show(dn);
     String d2 = new StateMachine<>(n)
          .next(main::a)
          .get();
     show(d2);
     Double d3 = new StateMachine<>(n)
          .next(main::a)
          .next(main::b)
          .get();
     show(d3);
     String d4 = new StateMachine<>(n)
          .next(main::a)
          .next(main::b)
          .next(main::c)
          .get();
     show(d4);
  }
        ……..
}
```

Есть клиентский код, необходимо реализовать остальную часть программы

```
public static void show(Object o) {
   System.out.println(o.getClass().getSimpleName() + ": " +o);
 }
 public State<String> a(State<Integer> num) {
   return State.of(String.valueOf(num.get()));
 }
 public State<Double> b(State<String> s) {
   return State.of(Double.parseDouble(s.get()) + 5.0);
 }
 public State<String> c(State<Double> d) {
   return State.of("number: " + d.get());
 }
```

Класс State должен быть generic классом:

```
public class State<T> {

   private T value;

   // TODO конструктор

   // TODO метод T get()

   // TODO статический метод <T> State<T> of(T value) {
      return вызов конструктора;
   }
}
```

Класс StateMachine также должен быть generic классом:

```
public class StateMachine<T> {
   private State<T> state;

   // TODO конструктор с аргументом T state
   // вызывает статический метод of класса State

   // TODO конструктор с аргументом State<T> state

  // TODO метод T get() который вызовет метод state.get()

   // TODO метод
   // <R> StateMachine<R> next (Function<State<T>, State<R>> function)
   // который вернет вызов конструктора StateMachine
   // и в аргументах будет вызов метода apply от аргумента function
   // и передавать туда будем state
}
```